Here are all 25 answers:

SET 1 — MEDIUM

Q1 [JOINs] *Write a query to find all customers who have placed at least one order.*

```
SELECT c.customer_name,
    COUNT(o.order_id) AS total_orders
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_name
ORDER BY total_orders DESC;
```

Concept: INNER JOIN returns only rows that match in both tables. COUNT(o.order_id) counts orders per customer — always count the foreign key not *.

Cross questions:

- What is the difference between INNER JOIN and LEFT JOIN?
- What if a customer has 0 orders — does INNER JOIN show them?
- What does COUNT(*) vs COUNT(column) return differently?

Tip: COUNT(*) counts all rows including NULLs. COUNT(column) skips NULLs. Always be explicit about which you mean.

Q2 [LEFT JOIN + NULL] *Find all customers who have never placed an order.*

```
SELECT c.customer_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Concept: LEFT JOIN keeps all customers even those with no orders. When no match exists, orders columns are NULL. WHERE o.order_id IS NULL isolates non-matching customers.

Cross questions:

- Why can't you use WHERE o.order_id = NULL?
- What is the difference between IS NULL and = NULL?
- Could you solve this with NOT IN or NOT EXISTS? Which is faster?

Tip: LEFT JOIN + WHERE right_table.key IS NULL is one of the most tested SQL patterns. Know it cold.

Q3 [GROUP BY + HAVING] *Find departments where average salary is greater than 60,000.*

```
SELECT department,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 60000
ORDER BY avg_salary DESC;
```

Concept: HAVING filters groups after aggregation. WHERE filters rows before aggregation. You cannot use WHERE with aggregate functions.

Cross questions:

- What is the difference between WHERE and HAVING?
- Can you use a column alias in HAVING?
- What is the order of SQL clause execution?

Tip: Order of execution — FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. Know this. It explains every SQL confusion.

Q4 [SUBQUERY] *Find employees who earn more than the average salary of their department*

```
SELECT employee_name, salary, department
FROM employees e
WHERE salary > (
   SELECT AVG(salary)
   FROM employees
   WHERE department = e.department
);
```

Concept: A correlated subquery runs once per row of the outer query. It references the outer query's current row via e.department.

Cross questions:

- What is the difference between a correlated and non-correlated subquery?
- Could you solve this with a window function instead? Which is more efficient?
- What happens if the subquery returns NULL?

Tip: Correlated subqueries are powerful but slow on large datasets. Show you know both the subquery and window function approach.

Q5 [CASE WHEN] *Classify employees as High, Medium or Low earner.*

```
SELECT employee_name,
    salary,
    CASE
      WHEN salary > 80000 THEN 'High'
      WHEN salary BETWEEN 50000 AND 80000 THEN 'Medium'
      ELSE 'Low'
    END AS salary_band
FROM employees;
```

Concept: CASE WHEN evaluates conditions in order and returns the first match. ELSE handles anything that does not match any condition.

Cross questions:

- What if you forgot the ELSE clause?
- Can you use CASE WHEN inside GROUP BY?
- Can you use CASE WHEN inside ORDER BY?

Tip: CASE WHEN inside SUM() creates conditional aggregates — SUM(CASE WHEN status='paid' THEN amount ELSE 0 END). This is extremely powerful and leads directly to Q16.

Q6 [WINDOW — ROW_NUMBER] *Assign a row number to each employee within their department ordered by salary descending.*

```
SELECT employee_name,
    department,
    salary,
    ROW_NUMBER() OVER (
      PARTITION BY department
      ORDER BY salary DESC
    ) AS row_num
FROM employees;
```

Concept: PARTITION BY resets the row number for each department. ORDER BY salary DESC numbers the highest earner as 1. ROW_NUMBER always gives unique numbers even for ties.

Cross questions:

- What is the difference between ROW_NUMBER, RANK and DENSE_RANK?
- How would you use this to find the top earner in each department?
- What happens if two employees have the same salary?

Tip: Wrap this in a CTE and add WHERE row_num = 1 to get the top earner per department. One of the most asked patterns in analytics interviews.

Q7 [RANK vs DENSE_RANK] *Show the difference between RANK() and DENSE_RANK().*

```
SELECT employee_name,
    salary,
    RANK()       OVER (ORDER BY salary DESC) AS rank_num,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank_num,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM employees;
```

Concept: If salaries are 90k, 90k, 80k:

- RANK gives 1, 1, 3 — skips after tie
- DENSE_RANK gives 1, 1, 2 — no skip
- ROW_NUMBER gives 1, 2, 3 — always unique

Cross questions:

- When would you use RANK vs DENSE_RANK in a real scenario?
- Find the second highest salary — which function would you use?
- What is the difference between OVER() and OVER(PARTITION BY department)?

Tip: The 1,1,3 vs 1,1,2 example is the single most asked SQL concept in analytics interviews. Memorise it.

Q8 [DATE FUNCTIONS] *Find orders in the last 30 days. Extract month and year.*

```
-- Last 30 days
SELECT order_id, order_date
FROM orders
WHERE order_date >= CURRENT_DATE - INTERVAL '30 days';

-- Extract month and year
```

```sql
SELECT order_id,
    EXTRACT(MONTH FROM order_date) AS order_month,
    EXTRACT(YEAR  FROM order_date) AS order_year
FROM orders;
```

Concept: CURRENT_DATE returns today. INTERVAL subtracts time periods. EXTRACT pulls specific parts from a date.

Cross questions:

- How do you calculate the number of days between two dates?
- What is the difference between DATE and DATETIME?
- How would you group orders by week?

Tip: Always clarify which SQL dialect — MySQL uses DATEDIFF, PostgreSQL uses INTERVAL, SQL Server uses DATEADD. Mention this to show awareness.

Q9 [STRING FUNCTIONS] *Find gmail.comcustomers. Trim spaces from names.*

```sql
SELECT TRIM(customer_name) AS clean_name,
    email
FROM customers
WHERE email LIKE '%@gmail.com';
```

Concept: LIKE with % matches any characters. TRIM removes leading and trailing spaces. LTRIM removes left only, RTRIM removes right only.

Cross questions:

- What is the difference between LIKE and ILIKE?
- How would you extract just the domain from an email address?
- What is the performance impact of LIKE '%value%' with wildcard on both sides?

Tip: LIKE with a leading wildcard cannot use an index and causes a full table scan. Always mention this in optimisation discussions.

Q10 [SELF JOIN] *Find employees who earn more than their manager.*

```sql
SELECT e.employee_name,
    e.salary       AS employee_salary,
    m.employee_name AS manager_name,
    m.salary       AS manager_salary
FROM employees e
```

```
JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

Concept: A self join joins a table to itself using aliases. e is the employee, m is the manager — same table joined twice.

Cross questions:

- What happens to the CEO who has no manager?
- How would you show all employees including those with no manager?
- What is a recursive CTE and when would you use it instead?

Tip: Use LEFT JOIN instead of JOIN if you want to include employees with no manager. Common follow-up.

SET 2 — CHALLENGING

Q11 [LAG/LEAD] *Month-over-month revenue growth.*

```
WITH monthly_revenue AS (
   SELECT DATE_TRUNC('month', order_date) AS month,
        SUM(amount) AS revenue
   FROM orders
   GROUP BY 1
)
SELECT month,
      revenue,
      LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,
      ROUND(
         (revenue - LAG(revenue) OVER (ORDER BY month))
         / LAG(revenue) OVER (ORDER BY month) * 100, 2
      ) AS mom_growth_pct
FROM monthly_revenue
ORDER BY month;
```

Concept: LAG() accesses the previous row's value. MoM % = (current - previous) / previous * 100. First month shows NULL — no previous month exists.

Cross questions:

- What does LEAD() do differently from LAG()?
- How would you handle the NULL in the first month?

- What is the difference between LAG(col, 1) and LAG(col, 2)?

Tip: LAG/LEAD MoM growth is tested in almost every senior analytics interview. Practice until it is second nature.

Q12 [RUNNING TOTAL] *Running total of sales by date.*

```sql
SELECT sale_date,
       daily_sales,
       SUM(daily_sales) OVER (
          ORDER BY sale_date
          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM daily_sales
ORDER BY sale_date;
```

Concept: SUM() as a window function with ORDER BY accumulates values from the first row to the current row. The frame clause makes this explicit.

Cross questions:

- What is the difference between ROWS and RANGE in window frame?
- How would you calculate a running total that resets each month?
- How would you calculate a 7-day rolling sum instead?

Tip: Without ORDER BY inside OVER(), SUM() gives the grand total for every row — a common mistake.

Q13 [CTE] *Top 3 highest paid employees per department.*

```sql
WITH ranked_employees AS (
   SELECT employee_name,
          department,
          salary,
          DENSE_RANK() OVER (
             PARTITION BY department
             ORDER BY salary DESC
          ) AS salary_rank
   FROM employees
)
SELECT employee_name, department, salary, salary_rank
FROM ranked_employees
WHERE salary_rank <= 3;
```

Concept: CTE creates a named intermediate result. You cannot use window functions directly in WHERE — the CTE wraps the window function so you can filter on it.

Cross questions:

- Why can't you use WHERE salary_rank <= 3 without a CTE?
- What is the difference between a CTE and a subquery?
- When would you chain multiple CTEs together?

Tip: Always use CTEs over nested subqueries in interviews. They show structured, readable thinking.

Q14 [MULTIPLE JOINs] *Order details with customer name and product name.*

```
SELECT o.order_id,
       c.customer_name,
       p.product_name,
       o.quantity,
       o.order_date
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN products  p ON o.product_id  = p.product_id
ORDER BY o.order_date DESC;
```

Concept: Multiple JOINs chain tables together. Orders is the fact table. Customers and Products are dimension tables.

Cross questions:

- What is a fact table vs a dimension table?
- How would you detect if a JOIN caused row duplication?
- What is a star schema vs a snowflake schema?

Tip: Always check row counts before and after multiple joins. Fan-out from unexpected duplicates is a very common real-world bug.

Q15 [DEDUPLICATION] *Keep only the most recent row per customer.*

```
WITH ranked_transactions AS (
   SELECT *,
          ROW_NUMBER() OVER (
             PARTITION BY customer_id
             ORDER BY transaction_date DESC
```

```
        ) AS rn
    FROM transactions
)
SELECT *
FROM ranked_transactions
WHERE rn = 1;
```

Concept: PARTITION BY customer_id groups duplicates. ORDER BY transaction_date DESC puts the most recent row first as rn=1. WHERE rn=1 keeps only the latest.

Cross questions:

- How would you DELETE duplicates from the original table?
- What if you want to keep the oldest record instead?
- How would you find the duplicates before deciding which to keep?

Tip: This ROW_NUMBER deduplication pattern is asked in almost every data engineering interview. Know it cold.

Q16 [CONDITIONAL AGGREGATION] *Total sales and quarterly breakdown in one row per product*

```
SELECT product_name,
    SUM(amount) AS total_sales,
    SUM(CASE WHEN EXTRACT(QUARTER FROM sale_date) = 1
        THEN amount ELSE 0 END) AS q1_sales,
    SUM(CASE WHEN EXTRACT(QUARTER FROM sale_date) = 2
        THEN amount ELSE 0 END) AS q2_sales,
    SUM(CASE WHEN EXTRACT(QUARTER FROM sale_date) = 3
        THEN amount ELSE 0 END) AS q3_sales,
    SUM(CASE WHEN EXTRACT(QUARTER FROM sale_date) = 4
        THEN amount ELSE 0 END) AS q4_sales
FROM sales
GROUP BY product_name;
```

Concept: SUM(CASE WHEN condition THEN value ELSE 0 END) is conditional aggregation. ELSE 0 ensures non-matching rows contribute 0 not NULL.

Cross questions:

- What is the difference between this and using PIVOT?
- What does COUNT(CASE WHEN...) do differently from SUM(CASE WHEN...)?
- How would you add a YoY comparison using this technique?

Tip: SUM(CASE WHEN) works in every database. PIVOT syntax varies. Always prefer CASE WHEN in interviews.

Q17 [FAN-OUT] *Demonstrate and fix the fan-out problem.*

```
-- Detect fan-out
SELECT COUNT(*) FROM orders;
SELECT COUNT(*) FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id;
-- If second count is higher -- fan-out!

-- Fix: aggregate BEFORE joining
WITH order_totals AS (
    SELECT order_id, SUM(amount) AS total_amount
    FROM order_items
    GROUP BY order_id
)
SELECT o.order_id, c.customer_name, ot.total_amount
FROM orders o
JOIN customers    c  ON o.customer_id = c.customer_id
JOIN order_totals ot ON o.order_id  = ot.order_id;
```

Concept: Fan-out occurs when JOIN multiplies rows due to one-to-many relationships. Fix by pre-aggregating the many-side table before joining.

Cross questions:

- How do you detect fan-out in a result set?
- Why does fan-out cause SUM() to give wrong results?
- What is a many-to-many relationship and how do you handle it?

Tip: Mentioning fan-out proactively in an interview shows you have worked with real data at scale. It impresses senior interviewers.

Q18 [NTILE] *Divide customers into 4 spend quartiles.*

```
SELECT customer_id,
       customer_name,
       total_spend,
       NTILE(4) OVER (ORDER BY total_spend ASC) AS spend_quartile
FROM (
    SELECT customer_id, customer_name,
```

```
        SUM(amount) AS total_spend
    FROM orders
    GROUP BY customer_id, customer_name
) customer_spend
ORDER BY total_spend DESC;
```

Concept: NTILE(4) divides customers into 4 equal groups. Quartile 1 = lowest spenders, Quartile 4 = highest spenders.

Cross questions:

- What is the difference between NTILE and PERCENT_RANK?
- How would you find the exact 90th percentile value?
- How would you label each quartile as Bronze, Silver, Gold, Platinum?

Tip: Combine NTILE with CASE WHEN to assign meaningful tier labels to each bucket.

SET 3 — ADVANCED

Q19 [RECURSIVE CTE] *All employees under a given manager — all levels.*

```
WITH RECURSIVE org_hierarchy AS (
    -- Anchor: start with the given manager
    SELECT employee_id, employee_name, manager_id, 1 AS level
    FROM employees
    WHERE employee_id = :manager_id

    UNION ALL

    -- Recursive: find all direct reports of current level
    SELECT e.employee_id, e.employee_name, e.manager_id, h.level + 1
    FROM employees e
    JOIN org_hierarchy h ON e.manager_id = h.employee_id
)
SELECT employee_id, employee_name, level
FROM org_hierarchy
ORDER BY level, employee_name;
```

Concept: Anchor query returns the root. UNION ALL combines with recursive member. Recursion stops when no more employees match. level tracks depth.

Cross questions:

- How do you prevent infinite loops in a recursive CTE?
- What is the maximum recursion depth in your SQL dialect?
- How would you display this as an indented org chart?

Tip: Recursive CTEs are tested in senior data engineering interviews. The org hierarchy example is the most common use case — practice it.

Q20 [PIVOT] *One row per product with monthly sales columns.*

```
SELECT product_name,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 1
        THEN amount ELSE 0 END) AS jan,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 2
        THEN amount ELSE 0 END) AS feb,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 3
        THEN amount ELSE 0 END) AS mar,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 4
        THEN amount ELSE 0 END) AS apr,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 5
        THEN amount ELSE 0 END) AS may,
    SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 6
        THEN amount ELSE 0 END) AS jun
FROM sales
GROUP BY product_name;
```

Concept: Conditional aggregation creates one column per month. Works in all databases — no native PIVOT keyword needed.

Cross questions:

- What is the native PIVOT syntax in SQL Server?
- How would you unpivot — columns back to rows?
- What if the number of months is dynamic?

Tip: Dynamic pivoting requires dynamic SQL. Mention that for truly dynamic cases, Python/Pandas is more appropriate. Shows business thinking.

Q21 [GAPS AND ISLANDS] *Start and end dates of each consecutive login streak.*

```
WITH login_data AS (
  SELECT user_id,
      login_date,
      ROW_NUMBER() OVER (
```

```
            PARTITION BY user_id
            ORDER BY login_date
        ) AS rn
    FROM user_logins
),
island_id AS (
    SELECT user_id,
        login_date,
        login_date - INTERVAL '1 day' * rn AS island
    FROM login_data
)
SELECT user_id,
    MIN(login_date) AS streak_start,
    MAX(login_date) AS streak_end,
    COUNT(*)       AS streak_length
FROM island_id
GROUP BY user_id, island
ORDER BY user_id, streak_start;
```

Concept: Subtracting row number from date gives the same constant for consecutive days. A gap causes this constant to change — creating a new island.

Cross questions:

- How would you find users with a streak of 7+ consecutive days?
- What if the table has duplicate entries per day?
- How does this work if dates are not stored as DATE type?

Tip: The date-minus-row-number trick is the most elegant gaps and islands solution. Interviewers are genuinely impressed when you know it.

Q22 [QUERY OPTIMISATION] *5 techniques for a slow query on 50M rows.*

```
-- 1. Add index on filter/join column
CREATE INDEX idx_orders_date ON orders(order_date);

-- 2. Partition pruning — query with partition key in WHERE
SELECT * FROM orders
WHERE order_date >= '2024-01-01'
  AND order_date <  '2025-01-01';

-- 3. Avoid SELECT * — select only needed columns
SELECT order_id, customer_id, amount
FROM orders WHERE order_date > '2024-01-01';
```

```sql
-- 4. Pre-aggregate before joining
WITH agg AS (
    SELECT customer_id, SUM(amount) total
    FROM orders GROUP BY customer_id
)
SELECT c.name, agg.total
FROM customers c JOIN agg ON c.id = agg.customer_id;

-- 5. Check query plan
EXPLAIN ANALYZE SELECT ...;
```

Concept: Optimisation order — diagnose first with EXPLAIN, then indexes, then query rewrite. Never jump to solutions before diagnosing.

Cross questions:

- What is the difference between a clustered and non-clustered index?
- When would an index actually slow a query down?
- What is query plan caching and when does it cause problems?

Tip: Always structure your optimisation answer: check the query plan first, then indexes, then rewrite. This structured approach impresses interviewers.

Q23 [MOVING AVERAGE] *7-day moving average of daily active users.*

```sql
SELECT activity_date,
       daily_active_users,
       AVG(daily_active_users) OVER (
          ORDER BY activity_date
          ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS rolling_7day_avg
FROM dau_metrics
ORDER BY activity_date;
```

Concept: ROWS BETWEEN 6 PRECEDING AND CURRENT ROW = 7 rows total. AVG over this window smooths out day-of-week effects.

Cross questions:

- What is the difference between ROWS and RANGE in the frame clause?
- How would you exclude the current row from the average?
- How would you calculate a 30-day moving average?

Tip: Mention that the first 6 days will have partial windows — fewer than 7 days available. Shows you think about edge cases.

Q24 [FIRST/LAST VALUE] *First and most recent product purchased per customer.*

```
SELECT DISTINCT
    customer_id,
    FIRST_VALUE(product_name) OVER (
        PARTITION BY customer_id
        ORDER BY purchase_date ASC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
FOLLOWING
    ) AS first_product,
    LAST_VALUE(product_name) OVER (
        PARTITION BY customer_id
        ORDER BY purchase_date ASC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
FOLLOWING
    ) AS last_product
FROM purchases;
```

Concept: LAST_VALUE needs UNBOUNDED FOLLOWING — without it, the default frame stops at the current row and gives wrong results.

Cross questions:

- Why does LAST_VALUE give wrong results without the explicit frame?
- How would you use a CTE approach instead?
- What if a customer has only one purchase?

Tip: LAST_VALUE is notoriously tricky because of the default frame. Always specify ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

Q25 [FULL OUTER JOIN + COALESCE]*Compare revenue across two years including products in only one year*

```
WITH revenue_2023 AS (
    SELECT product_id, SUM(amount) AS rev_2023
    FROM sales WHERE EXTRACT(YEAR FROM sale_date) = 2023
    GROUP BY product_id
),
revenue_2024 AS (
    SELECT product_id, SUM(amount) AS rev_2024
    FROM sales WHERE EXTRACT(YEAR FROM sale_date) = 2024
```

```
    GROUP BY product_id
)
SELECT
    COALESCE(a.product_id, b.product_id) AS product_id,
    COALESCE(a.rev_2023, 0)              AS revenue_2023,
    COALESCE(b.rev_2024, 0)              AS revenue_2024,
    COALESCE(b.rev_2024, 0)
    - COALESCE(a.rev_2023, 0)            AS yoy_change
FROM revenue_2023 a
FULL OUTER JOIN revenue_2024 b ON a.product_id = b.product_id
ORDER BY yoy_change DESC;
```

Concept: FULL OUTER JOIN includes products from both years. COALESCE replaces NULL with 0 for products that exist in only one year.

Cross questions:

- What is the difference between FULL OUTER JOIN, LEFT JOIN and INNER JOIN?
- When would COALESCE give a different result from ISNULL?
- How would you extend this to compare 3 years?

Tip: FULL OUTER JOIN + COALESCE is the standard YoY comparison pattern. Asked heavily in finance and e-commerce analytics interviews.